“Who do you think the enemy is?”

Attack on Titan — The Battle for Wall Rose

Authors: **Lakshya Tambi, Arnav, Rohan**

Duration	90 Minutes
Files	TitanShifter.java ScoutSquad.java WallDistrict.java BattleOfWallRose.java

Scenario

A big wall that protects a city has been broken, and giant monsters called **Titans** are coming in. Squads of soldiers must defend the city.

Some people, called **Titan Shifters**, can turn into a Titan. A Shifter is either **hostile** (an enemy) or **friendly** (on our side). A friendly Shifter can also give commands to the ordinary Titans, and this ability is called **Coordinate**.

Soldiers send each other messages written in binary and hexadecimal, and these messages must be decoded. Each squad checks whether it is strong enough to fight a hostile Shifter. If it is not, it asks a friendly Shifter for help.

Your task: complete the four Java classes described below so that the whole simulation compiles and runs correctly. **The getters and setters of TitanShifter, ScoutSquad and BattleOfWallRose, and the non-static inner class TitanForm, are already provided in the skeleton code. Do not change them. Implement everything marked `TODO`, including all the getters and setters of WallDistrict.**

Class 1 — TitanShifter

Models a Titan Shifter tracked by military intelligence. Wrapper types are used deliberately throughout — understand how they behave during autoboxing, unboxing, and re-boxing.

Fields

String name | **Integer** powerLevel | **Double** heightMeters | **Boolean** isHostile

Constructors (Overloaded)

- TitanShifter(String name, String powerStr, String heightStr, String hostileStr) — Parse each string into its corresponding wrapper type using the appropriate valueOf() method.
- TitanShifter(String name, Integer powerLevel, Double heightMeters, Boolean isHostile) — Directly assign the typed parameters to the corresponding fields.

Getters & Setters (Provided)

The standard getters and setters for all four fields (getName/setName, getPowerLevel/setPowerLevel, getHeightMeters/setHeightMeters, getIsHostile/setIsHostile) are already implemented. **Nothing needs to be changed here.**

Non-Static Inner Class — TitanForm (Provided)

The inner class TitanForm is already fully implemented in the skeleton code. **Do NOT change anything in it.** It is described here only so that you understand how it is used by BattleOfWallRose:

- public static final int MAX_TRANSFORM_LIMIT = 3; — Maximum transformations allowed before stamina depletion.
- TitanForm(String transformCountStr) — Parses transformation count from its string representation.
- TitanForm(int transformCount) — Overloaded constructor.
- getTransformCount() and setTransformCount(Integer count).

- Integer getPowerSurge() — Computes combat power surge as powerLevel × transformCount × 2 using the enclosing shifter's powerLevel.

Static Nested Class — CoordinatePower

Tracks activeTitansControlled as a static Integer. Its getter and setter, static Integer getActiveTitansControlled() and static void setActiveTitansControlled(Integer count), are provided (nothing to change). You only need to implement:

- static Boolean triggerCoordinate(TitanShifter shifter, String commandSignal) — Decodes binary command (radix 2). If the shifter is friendly (not hostile) and the decoded signal is greater than 0, sets activeTitansControlled = decodedSignal × 10 and returns true. Otherwise returns false.

Method — assessCombatReadiness (TODO)

static int[] assessCombatReadiness(Integer power) — Increases the given power rating by 1. Using this updated power, computes *score* (stored as an Integer) and *rawScore* (stored as an int), each equal to power + (power × 3). Returns an int array [score, rawScore].

Class 2 — ScoutSquad

Models an elite combat squad in the Survey Corps. Uses overloaded constructors where one constructor calls another in the same class using this(...) (constructor chaining), varargs, and Boolean wrapper types.

Fields

String squadLeader | **String[]** members | **Integer** squadPower | **Boolean** deployed

Constructors (Overloaded & Varargs)

- ScoutSquad(String leader) — Single-leader squad. Calls the second constructor using this(...) by passing leader and an empty String array (constructor chaining).
- ScoutSquad(String leader, String... members) — Varargs constructor that assembles a squad with a leader and squad members. The varargs parameter may be empty (no squad members). Computes squadPower = members.length × 50 + 100 as an Integer. Sets deployed to the Boolean representation of false (the squad is not deployed initially).

Getters & Setters (Provided)

Getters and setters for all four fields (getSquadLeader/setSquadLeader, getMembers/setMembers, getSquadPower/setSquadPower, isDeployed/setDeployed) are already implemented. **Nothing needs to be changed here.**

Methods

- void boostSquadPower(Integer bonus) — Boosts squad power by the given bonus amount.
- boolean canEngage(TitanShifter titan) — Returns true only if the titan is hostile and squadPower ≥ titan.getPowerLevel().
- String[] getFullRoster() — Returns an array with the leader at index 0 followed by all members.

Class 3 — WallDistrict

Represents a fortified district along Wall Rose (e.g. Trost, Krolva, Ehrmich). Handles district kill matrices, signal decoding, and titan identification.

Fields

String districtName | **Integer** wallIntegrity | **Integer[][]** titanKillMatrix | **TitanShifter** spottedTitan

Sector & Squad Matrix: Rows represent defence sectors along the district wall (Sector 0, Sector 1, …), columns represent assigned defence squads (Squad 0, Squad 1, …). For example, matrix[0][1] = 5 means Sector 0 had 5 confirmed kills credited to Squad 1. A null entry means that squad was not deployed there.

Constructors (Overloaded)

- WallDistrict(String name) — Creates a district with default values: districtName is set to name; wallIntegrity is set to the Integer 100 (i.e. 100%); titanKillMatrix is set to an empty 2D array — an actual array object with zero rows (e.g. new Integer[0][0]), not null; spottedTitan is null.
- WallDistrict(String name, String integrityStr, Integer[][] killMatrix) — districtName is set to name; integrityStr (e.g. "85") is parsed into an Integer using Integer.valueOf() and stored in wallIntegrity; the given killMatrix is assigned to titanKillMatrix as it is (it may contain null entries, and its rows may have different lengths); spottedTitan starts as null.

Getters & Setters (TODO)

Implement the standard getter and setter for all four fields (getDistrictName/setDistrictName, getWallIntegrity/setWallIntegrity, getTitanKillMatrix/setTitanKillMatrix, getSpottedTitan/setSpottedTitan).

Methods

- Integer calculateTotalConfirmedKills() — Adds up every kill count stored in titanKillMatrix and returns the total as an Integer. Use nested loops: the outer loop goes over each row (sector) and the inner loop goes over each entry (squad) in that row. Because the matrix is of type Integer[][], an entry can be null (squad not deployed); a null entry must be skipped, since unboxing a null Integer while adding would throw a NullPointerException. Use each row's own length (matrix[i].length) for the inner loop. An empty matrix gives a total of 0.

Example: for the matrix below, the total is 3 + 2 + 5 + 1 = 11 (the two null entries are skipped).

	Squad 0	**Squad 1**	**Squad 2**
Sector 0	3	null	2
Sector 1	null	5	1

- boolean verifyTitanIdentification(String reportId1, String reportId2) — Parses both report IDs to Integer and verifies whether they identify the same titan.
- int[] decodeSignal(String binarySignal, String hexSignal) — Decodes binary signal (base 2) and hex signal (base 16). Returns [urgencyLevel, titanCount].

Class 4 — BattleOfWallRose (Master Coordinator)

The master tactical command system. Maintains arrays of registered WallDistrict, ScoutSquad, and TitanShifter objects (max 5, 10, and 10 respectively) and a running totalHumanityDefenseScore (Integer). The final method executeDefenseOperation ties together every class you have written.

Fields

WallDistrict[] districts | **ScoutSquad[]** activeSquads | **TitanShifter[]** encounteredTitans | **Integer** totalHumanityDefenseScore

Constructor & Registration Methods

- BattleOfWallRose() — Initialise all three arrays to their max sizes; set all counters and totalHumanityDefenseScore to 0.
- boolean addDistrict(WallDistrict district), boolean registerSquad(ScoutSquad squad), boolean registerTitan(TitanShifter titan) — Add the given object to its respective array if capacity is not exceeded and the object is not null; return true on success.

Getters (Provided)

getTotalHumanityDefenseScore(), getDistrictCount(), getSquadCount() and getTitanCount() are already implemented. **Nothing needs to be changed here.**

Overloaded Methods (Varargs)

- static int deploySquadUnits(int squadSize) — Exact-match overload; returns squadSize.

- static int deploySquadUnits(int firstSquad, int... otherSquads) — Varargs overload; returns the sum of all squad sizes.

Helper Method

- TitanShifter findFriendlyShifter() — Searches encounteredTitans for a shifter whose isHostile is false. Returns the first match, or null if none found.

Master Execution Method — executeDefenseOperation

boolean executeDefenseOperation(String commandSignal, String hexIntel)

Orchestrates the entire operation across all registered districts:

Note: Steps 1, 2, 3b and 4 below are already provided in the skeleton code, so you only need to implement step 3a. You might have to make a few changes in the provided code.

1. **Signal Intelligence (provided):** For each district, call district.decodeSignal(commandSignal, hexIntel) and add the decoded urgency level and titan count to totalHumanityDefenseScore.

2. **Kill Tallying (provided):** Call district.calculateTotalConfirmedKills() and add the result to the score.

3a. **Hostile Titan Spotted (YOU IMPLEMENT):** Search activeSquads for a non-deployed squad that canEngage the titan. If found, mark it deployed and add its squadPower to the score. If no squad can engage alone, find a friendly shifter via findFriendlyShifter(), create a TitanForm (count "2"), retrieve its getPowerSurge() to boost the first undeployed squad's power, mark it deployed, add its updated power to the score, and also run TitanShifter.assessCombatReadiness(friendly.getPowerLevel()) and add result[0] to the score.

3b. **Friendly Titan Spotted (provided):** Call TitanShifter.CoordinatePower.triggerCoordinate(spottedTitan, commandSignal). If it returns true, add CoordinatePower.getActiveTitansControlled() to the score.

4. **Outcome (provided):** Return true if totalHumanityDefenseScore ≥ 100 (Wall Rose secured); otherwise return false.

All the best!